Python User Defined Functions

A function is a block of code which only runs when it is called. It is useful when a set of code has t be executed repeatedly.

We can pass data, known as parameters, into a function.

A function can return data as a result.

Creating a Function

In Python a function is defined using the **def** keyword:

Types of Function

1) Simple Function
2) Function with Arguments
3) Return type function

Example

```
def my_function():
  print("Hello Python")
```

And at the end insert 2 line breaks.

Calling a Function

To call a function, use the function name followed by parenthesis:

Example

```
def my_function():
  print("Hello Python")
my_function()
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, separated by a comma.

The following example has a function with one argument (city). When the function is called, we pass along a city name, which is used inside the function to print the city:

Example

```
def my_function(city):
  print(city)

my_function("Jaipur")
my_function("Mumbai")
my_function("Chennai")
```

Number of Arguments/ Positional Arguments

By default, a function must be called with the correct number of arguments. That is if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

Example

This function expects 2 arguments, and gets 2 arguments:

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("New", "Delhi")
```

Arbitrary Arguments, *args

If you do not know, how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a tuple of arguments, and can access the items accordingly:

Example

If the number of arguments is unknown, add a * before the parameter name:

```
def my_function(*city):
  print("The last city is " + city[2])

my_function("New Delhi", "Kochi", "Bhopal")
```

Keyword Arguments

You can also send arguments with the key = value syntax.

This way the order of the arguments does not matter.

Example

```
def my_function(city3, city2, city1):
  print("The first city is " + city1)



my_function(city1 = "Mumbai", city2 = "Chennai", city3 = "Jaipur")
```

Arbitrary Keyword Arguments, **kwargs

If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ** before the parameter name in the function definition.

This way the function will receive a dictionary of arguments, and can access the items accordingly:

Example

If the number of keyword arguments is unknown, add a double ** before the parameter name:

```
def my_function(**city):
```

```
  print("City last name is " + city["lname"])


my_function(fname = "New", lname = "Delhi")
```

Default Parameter Value

If we call the function without argument, it uses the default value:

Example

```
def my_function(country = "India"):
    print("I am from " + country)



my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

Example

```
def my_function(fruit):
  for x in fruit:
    print(x)

fruits = ["apple", "banana", "cherry"]
my_function(fruits)
```

Return Values

To return a value from function, use the return statement:

Example

```
def my_function(x):
    return 5 * x



print(my_function(3))
print(my_function(5))
print(my_function(9))
```

The pass Statement

function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

Example

```
def myfunction():
  pass
```

Positional-Only Arguments

You can specify that a function can have ONLY positional arguments, or ONLY keyword arguments.

To specify that a function can have only positional arguments and not keyword arguments, add **, /** after the arguments:

Example

```
def my_function(x, /):
   print(x)
```

my_function(3)Without the , / you are actually allowed to use keyword arguments even if the function expects positional arguments:

Example

```
def my_function(x):
  print(x)
```

```
my_function(x = 3)
```

But when adding the , / you will get an error if you try to send a keyword argument:

Example

```
def my_function(x, /):
  print(x)
```

```
my_function(x=3)
```

Keyword-Only Arguments

To specify that a function can have only keyword arguments, add *, before the arguments:

Example

```
def my_function(*, x):
  print(x)

my_function(x = 3)
```

Without the *, you are allowed to use positionale arguments even if the function expects keyword arguments:

Example

```
def my_function(x):
  print(x)

my_function(3)
```

But when adding the *, / you will get an error if you try to send a positional argument:

Example

```
def my_function(*, x):
  print(x)

my_function(3)
```

Combine Positional-Only and Keyword-Only

You can combine the two argument types in the same function.

Any argument before the / , are positional-only, and any argument after the *, are keyword-only.

Example

```
def my_function(a, b, /, *, c, d):
    print(a + b + c + d)

my_function(5, 6, c=7, d=8)
```

Recursion

Python also accepts function recursion, which means a defined function can call itself.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

The developer should be very careful with recursion as it can be quite easy to slip into writing a function which **never terminates**, or one that uses excess amounts of memory or processor power. However, when written correctly recursion can be a very efficient and mathematically-elegant approach to programming.

In this example, tri_recursion() is a function that we have defined to call itself ("recurse"). We use the k variable as the data, which decrements (-1) every time we recurse. The recursion ends when the condition is not greater than 0 (i.e. when it is 0).

To a new developer it can take some time to work out how exactly this works, best way to find out is by testing and modifying it.

Recursion Example

```
def tri_recursion(k):
    if (k > 0):
        k = k - 1
        print(k)
        tri_recursion(k)
    else:
        result = 0
    return k


tri_recursion(6)
```

Python File I/O

Files are used to store data in a computer disk. We can open a file using Python script and perform various operations such as writing, reading, and appending.

Python's file input/output (I/O) system offers programs to communicate with files stored on a disc. Python has built-in methods for the file object

The open() method in Python makes a file object when working with files.

The name of the file to be opened and the mode in which the file is to be opened are the two parameters required by this function.

The mode can be used according to work that needs to be done with the file, such as **"r" for reading**, **"w" for writing**, or **"a" for attaching**.

The following modes are supported:

r: open an existing file for a read operation.

w: open an existing file for a write operation. If the file already contains some data, then it will be overridden but if the file is not present then it creates the file as well.

a: open an existing file for append operation. It won't override existing data.

r+: To read and write data into the file. The previous data in the file will be overridden.

w+: To write and read data. It will override existing data.

a+: To append and read data from the file. It won't override existing data.

Binary files contains data in a binary rather than a text format may also be worked with using Python. Binary files are written in a manner that humans cannot directly understand. The **rb** and **wb** modes can read and write binary data in binary files.

Python Text file

Working in Write Mode

Open a file to write

Write your text

Close the file

```
file = open('try.txt','w')
file.write("This is the write command")
file.write("It allows us to write in a particular file")
file.close()
```

Working in Read Mode

Open the file

Read the file

No need to close

```
file = open('try.txt', 'r')

# This will print every line one by one in the file
for each in file:
```

```
    print (each)
```

Example 2: In this example, we will extract a string that contains all characters in the Python file then we can use file.read().

```
file = open("try.txt", "r")
print (file.read())
```

Example 3: In this example, we will see how we can read a file using the with statement in Python.

```
with open("try.txt") as file:
    data = file.read()

print(data)
```

Example 4: Another way to read a file is to call a certain number of characters like in the following code the interpreter will read the first five characters of stored data and return it as a string:

```
file = open("try.txt", "r")
print (file.read(5))
```

Output:

Example 5: We can also split lines while reading files in Python. The split() function splits the variable when space is encountered. You can also split using any characters as you wish.

```
with open("try.txt", "r") as file:
    data = file.readlines()
    for line in data:
        word = line.split()
        print (word)
```

Example 2: We can also use the written statement along with the with() function.

```
with open("file.txt", "w") as f:
    f.write("Hello World!!!")
```

Working in Append Mode

Open a file to append

append your text

Close the file

```
file = open('try.txt', 'a')

file.write("\nlets add this line")

file.close()
```